

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Nethra P(192511012) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

nethra

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

Constraints:

- i. D1 cannot work Night shift
- ii. D2 must work before D3 (Morning < Afternoon < Night)
- iii. Only one doctor per shift
- iv. D3 cannot work Morning
- v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

- i. Movement allowed: Up, Down, Left, Right
- ii. Cost of each move = 1
- iii. Cannot pass through obstacles
- iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

- a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

- b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
- c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
 - Rationality
 - Environment type
- d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Question 1 — AI-Powered Emergency Drones

(Greedy Best-First Search vs A*)

i. Behaviour of Greedy Best-First Search and its strengths/weaknesses under uncertainty

Greedy Best-First Search (GBFS) selects the next node to expand purely on the basis of the heuristic function $h(n)$, which in this case is the straight-line distance from the drone's current position to the delivery point. It ignores the actual path cost $g(n)$ accumulated so far. In the flood-affected scenario, this means the drone will always move toward the node that appears geographically closest to the destination, without considering whether the terrain along that direction is slower, more dangerous, or currently blocked.

Strengths: GBFS is computationally fast and has low memory overhead because it maintains a small frontier and expands very few nodes compared to exhaustive search. In a time-critical emergency, this speed is attractive because the drone can make a decision almost instantly. It is also easy to implement and works reasonably well when the heuristic closely reflects real travel difficulty.

Weaknesses under uncertainty: Because GBFS is not optimal and not complete in general graphs, it can be seriously misled in this dynamic environment. If the straight-line-nearest route suddenly becomes blocked by floodwater, GBFS has no mechanism to reconsider the true cost already spent — it may repeatedly dive toward a locally 'close' node that turns out to be a dead end, wasting critical time. It does not account for variable terrain/weather cost, so it may choose a route that looks short in distance but is actually the slowest or riskiest in practice. This makes GBFS unreliable when safety and guaranteed delivery matter more than raw speed of decision-making.

ii. How A* would handle the situation and balance cost and heuristic

A* Search evaluates every node using the function $f(n) = g(n) + h(n)$, where $g(n)$ is the actual accumulated cost from the start to node n (capturing real terrain and weather-based movement cost) and $h(n)$ is the straight-line-distance heuristic estimate to the goal. By combining both terms, A* does not simply chase the node that looks closest on the map; it balances 'how much has this route already cost me' against 'how much more do I optimistically expect to spend'.

In the drone-delivery scenario, this means A* will naturally avoid a route that looks short in a straight line but has high real movement cost due to flooded or rough terrain, because the $g(n)$ component penalises it. At the same time, because $h(n)$ is admissible (straight-line distance never overestimates true distance), A* remains guided toward the goal and does not explore irrelevant directions the way an uninformed search like uniform-cost search would. As real-time updates report a blocked path, A* can update the cost graph and re-plan, expanding the next-lowest $f(n)$ node from the frontier, effectively re-routing intelligently rather than blindly persisting on the previously 'closest-looking' path.

iii. Impact of heuristic accuracy on both algorithms

Heuristic accuracy is central to the performance of both algorithms, but it affects them differently. For Greedy Best-First Search, the quality of the solution depends almost entirely on the heuristic, since path cost is ignored completely — an inaccurate or misleading heuristic (e.g., straight-line distance across a flooded valley that is impassable) can cause GBFS to select very poor, even catastrophically inefficient, routes with no way to self-correct.

For A*, heuristic accuracy affects efficiency rather than correctness, provided the heuristic remains admissible (never overestimates) and, ideally, consistent. A highly accurate heuristic that stays close to the true remaining cost allows A* to expand very few nodes and converge quickly to the optimal route — essentially behaving like an efficient, guided search. A weak or loose heuristic (e.g., zero heuristic, which reduces A* to Dijkstra's/uniform-cost search) still guarantees an optimal path, but at the cost of expanding many more nodes and taking longer, which matters in an urgent delivery context. Thus, a poor heuristic degrades GBFS's solution quality directly, while it degrades A*'s speed but preserves optimality/correctness as long as admissibility holds.

iv. Effect of dynamic changes (blocked paths) on search performance

Both algorithms are traditionally designed for static, fully known environments, so dynamic blockage of roads introduces significant challenges. When a path becomes inaccessible mid-flight, any previously computed plan that relied on it becomes invalid, and the search must be repeated or repaired using the new information.

For GBFS, dynamic changes are especially damaging because the algorithm has no accumulated-cost bookkeeping to fall back on; when its greedily-chosen route is blocked, it must restart heuristic-driven expansion from the current node, which can lead to oscillation or backtracking with no guarantee of improvement. For A*, dynamic changes require either a full re-run of the search from the drone's current position with the updated cost graph, or the use of incremental/real-time variants such as D* Lite or Lifelong Planning A*, which efficiently repair only the affected part of the search tree rather than recomputing everything from scratch. In general, more frequent and unpredictable blockages increase the branching factor of effective replanning and the number of node expansions, degrading both response time and, for greedy methods, solution quality.

v. Recommendation and justification

A* Search (ideally an incremental/real-time variant such as D* Lite) is the most suitable choice for this scenario. Although GBFS offers faster raw decision speed, its lack of cost-awareness makes it unsafe and unreliable when routes can suddenly change due to flooding — a wrong greedy choice could delay delivery of emergency medicine or endanger the drone. A* offers a principled balance: it uses the heuristic to remain fast and goal-directed while still respecting real, variable terrain and weather costs, and it guarantees optimality when the heuristic is admissible. For a mission where both real-time constraints and safety/optimality matter, A* (enhanced with real-time replanning to handle partial observability and blocked paths) gives the best trade-off between responsiveness and reliability.

Question 2 — Smart City Traffic Signal Optimization (Local Search)

i. How Hill Climbing would work and its limitations

Hill Climbing starts with an initial signal-timing configuration and iteratively moves to a neighbouring configuration (e.g., increasing or decreasing green time at one intersection by a small increment) whenever that neighbour has a better (lower-cost) evaluation than the current state. It keeps only the current state in memory, making it extremely lightweight — well suited to continuously re-optimizing hundreds of intersections in real time.

However, Hill Climbing suffers from several well-known limitations. It is a purely greedy, locally-focused method, so it terminates as soon as no neighbouring move improves the objective, even if that point is far from the true global optimum. It has no mechanism to look ahead or accept a temporarily worse move to escape a poor region, and it does not maintain any search history, so it cannot backtrack. In a traffic network with many interacting intersections, adjusting one signal can create ripple effects elsewhere, and Hill Climbing's narrow, single-step-at-a-time view makes it very prone to settling in a poor configuration.

ii. Local maxima, plateaus and ridges with real-world interpretation

- **Local maxima/minima:** A configuration where every small adjustment to any single intersection's timing makes total waiting time worse, even though a very different, more coordinated configuration elsewhere in the space would perform much better. For example, optimizing one busy junction in isolation may look ideal locally, while a city-wide re-timing (e.g., a coordinated 'green wave') would perform far better but cannot be reached by single small steps.
- **Plateaus:** A flat region where many neighbouring timing configurations produce almost identical waiting times/congestion levels, so Hill Climbing cannot tell which direction is beneficial and may wander or stop prematurely. This happens at moderate traffic volumes where small timing changes have negligible effect until a threshold is crossed.
- **Ridges:** A sequence of states where the optimal path to improvement requires simultaneous, coordinated changes at several intersections (e.g., adjusting offsets across a corridor together), but each individual single-intersection move looks like a step downhill. Hill Climbing, which only changes one variable at a time, cannot navigate such a ridge efficiently and may zig-zag without real progress.

iii. How Simulated Annealing or Genetic Algorithms overcome these limitations

Simulated Annealing (SA) introduces controlled randomness: at each step it may accept a worse-performing neighbouring configuration with a probability that depends on how much worse it is and on a 'temperature' parameter that gradually decreases over time. Early in the search (high temperature), SA freely accepts uphill (worse) moves, allowing it to escape local minima and cross plateaus/ridges; as temperature cools, it increasingly behaves like Hill Climbing, converging on a good configuration. This makes SA well suited to continuous re-optimization of traffic signals where escaping poor local configurations is essential.

Genetic Algorithms (GA) maintain a population of candidate signal-timing configurations rather than a single state. Through selection (favouring configurations with lower congestion cost), crossover (combining timing patterns from two good 'parent' configurations, e.g., merging good corridor coordination from one with good single-junction timing from another), and mutation (randomly perturbing some timings), GA explores many regions of the search

space in parallel. This population-based, stochastic exploration makes GA naturally resistant to getting trapped in a single local optimum, and it is well suited to discovering coordinated, ridge-like improvements across multiple intersections simultaneously.

iv. Recommended approach and justification (scalability and adaptability)

A hybrid approach is recommended: use a Genetic Algorithm (or a population-based metaheuristic) periodically/offline to discover strong, well-coordinated baseline timing plans for groups of intersections (corridors/zones), since GA scales well by parallelizing evaluation across a population and can capture ridge-like, multi-intersection coordination. Layer this with Simulated Annealing (or a lightweight local-search refinement) for continuous, real-time fine-tuning of individual intersections as traffic conditions fluctuate, since SA is computationally cheaper per step than running a full GA continuously and can quickly adapt a good baseline to short-term changes.

This combination is justified because GA provides the scalability and global exploration needed for the enormous, combinatorial search space of hundreds of intersections, while SA provides the low-overhead adaptability needed to react to constantly changing, real-time traffic flow without recomputing an entire population-based search from scratch. Pure Hill Climbing is rejected as the primary method because its susceptibility to local maxima, plateaus, and ridges would leave significant, avoidable congestion and fuel waste on the table in a system of this scale.

Question 3 — Mars Rover Exploration (Online Search Agents)

i. Why this problem requires an online search agent instead of offline search

Offline search algorithms (such as standard A* or BFS) assume the agent has complete knowledge of the state space, all actions, and their outcomes before execution begins, and they compute an entire solution path in advance before taking any action. On Mars, the terrain map is not known in advance — craters, rocks, and hazards are discovered only as the rover's sensors reach them. This means the rover cannot pre-compute a full safe path from start to goal offline; it must interleave computation with actual movement, sensing the environment after each action and deciding the next action based on what it has just observed.

This interleaving of acting and sensing is precisely the defining characteristic of an online search agent. The rover effectively 'discovers' the search graph as it explores it, and it must be able to act reasonably even though its knowledge of the world is incomplete — a requirement that offline search, which needs the full problem specification upfront, cannot satisfy.

ii. Challenges of partial observability and dynamic environments

Partial observability means the rover's sensors reveal only the local vicinity (e.g., a limited camera/lidar range), so it cannot know whether a promising-looking direction leads to a dead end, crater, or unsafe drop until it physically gets close enough to sense it. This can lead to costly backtracking, and in the worst case, an online agent might be led into positions from which it cannot safely return (a real risk when hazards are irreversible, such as toppling into a crater).

Additional challenges include: the possibility of re-visiting already-explored regions because the rover initially lacked information to know they were unhelpful; the difficulty of guaranteeing completeness or optimality, since a purely reactive online agent might not find the best path even if one exists, because it cannot see far enough ahead; and increased computational/communication burden, since decisions often need to be made with minimal or delayed input from mission control on Earth (light-time delay of several minutes each way), so the rover must largely reason autonomously and conservatively around uncertainty.

iii. How the rover builds and updates its internal model

The rover maintains an internal representation of the environment — often a grid-based or graph-based occupancy/hazard map — that starts largely unknown or assumed traversable. As the rover moves and its sensors return new readings, it updates this internal model incrementally: cells or regions found to contain rocks, craters, or steep slopes are marked as obstacles/hazards, safely traversed regions are marked as known-free, and unexplored regions remain marked as unknown.

This is analogous to the way online search algorithms like Learning Real-Time A* (LRTA*) update state-cost estimates each time a new area is explored: the rover revises its heuristic/cost estimates for nearby unexplored cells based on the newly sensed information, then re-plans its next local move using this updated model rather than a static, pre-known map. Over the course of exploration, the internal model becomes progressively more accurate,

allowing the rover to avoid previously-encountered hazards and to make increasingly informed decisions, effectively 'learning' the map as it explores it.

iv. Comparison of online search with uninformed and informed (offline) search strategies

- Knowledge requirement: Uninformed search (e.g., BFS, DFS) and informed offline search (e.g., A*) both require a fully specified state space in advance; online search operates with an initially unknown environment that is revealed through actual movement and sensing.
- Action-computation relationship: Offline methods compute the complete solution before any action is taken; online search interleaves computation and action, deciding one step at a time.
- Optimality/completeness guarantees: Offline informed search (A*) guarantees an optimal path given an admissible heuristic and full knowledge; online search generally cannot guarantee optimality (and in adversarial/unknown terrains, cannot even guarantee completeness) because decisions are made with partial information and may require costly backtracking.
- Adaptability: Uninformed and standard offline informed search cannot adapt if the environment changes after planning; online search is inherently adaptive because it continuously re-senses and re-plans, which is essential for a rover on genuinely unknown terrain.
- Risk handling: Offline search treats all edges as safely traversable if included in the plan; online search must be conservative about irreversible actions (e.g., not stepping toward an unseen cliff) since a wrong action cannot always be undone.

v. Justification of the most suitable approach

An online, informed local-search strategy — such as Learning Real-Time A* (LRTA*) combined with sensor-based hazard mapping — is the most suitable approach for the Mars rover. It is justified because the rover must operate under genuine uncertainty (unknown terrain), partial observability (limited sensor range), and the need for continuous adaptability (hazards discovered only during exploration). Pure uninformed or classical offline informed search is unusable here since the complete map does not exist beforehand. An online approach lets the rover safely and incrementally build knowledge of its surroundings, prioritize caution around irreversible hazards, and still use heuristic guidance (e.g., estimated distance/direction to the sample-collection goal) to keep exploration efficient rather than purely random, giving the best balance of safety, adaptability, and progress toward the mission objective.

Question 4 — University Exam Timetabling as a CSP

i. Variables, domains, and constraints

Variables:

- One variable per course/subject exam to be scheduled, e.g., Exam(CSA17), Exam(MAT21), etc. (Optionally, separate variables can represent the time slot, hall, and invigilator assignment for each exam, or these can be combined into a single composite variable per exam.)

Domains:

- For each exam variable, the domain is the Cartesian product of available time slots (e.g., each day/session across the exam period) and available exam halls (e.g., Hall-1, Hall-2, ..., Hall-N), representing every legal (slot, hall) combination that exam could be assigned to.

Constraints:

- Unary constraints: An exam requiring special accommodation must be assigned only to a hall equipped for that accommodation (e.g., ground-floor access, extra-time session slot).
- Binary/n-ary constraints: No two exams that share at least one common student may be assigned the same time slot (no overlapping exams for any student). No two exams may be assigned the same hall in the same time slot (hall-capacity/availability constraint). If a subject A must be scheduled before subject B, then $\text{slot}(A) < \text{slot}(B)$ (precedence constraint). A faculty member invigilating/setting an exam cannot be assigned to two exams in the same slot (faculty availability constraint). Hall capacity must be greater than or equal to the number of students registered for that exam (resource constraint).

ii. How backtracking search works in this scenario

Backtracking search assigns exams to (slot, hall) values one at a time, following a depth-first approach: it selects an unassigned exam, tries a value from its domain, and checks whether this assignment is consistent with all constraints involving previously assigned exams (e.g., no student clash, no hall clash, precedence maintained, faculty availability respected). If the assignment is consistent, the algorithm recursively moves on to assign the next exam. If at any point no value in the current exam's domain satisfies the constraints given the prior assignments, the algorithm backtracks — it undoes the most recent assignment and tries a different value, or backtracks further if that variable's domain is exhausted.

This systematic trial-and-undo process continues until either a complete, constraint-satisfying timetable is found (a full assignment of all exams to slots/halls) or the search proves that no solution exists with the current domains, at which point relaxation of soft constraints might be required.

iii. How constraint propagation (e.g., forward checking) improves efficiency

Plain backtracking can be inefficient because it often discovers a conflict only very late — after many exams have already been assigned — forcing extensive backtracking. Constraint propagation techniques address this by proactively reducing domains as assignments are made, catching conflicts earlier.

Forward checking, specifically, examines every not-yet-assigned exam that is directly constrained by the exam just assigned (e.g., exams sharing students, sharing a hall/slot, or under a precedence relation) and removes from their domains any (slot, hall) values that would now violate a constraint. If this causes any variable's domain to become

empty, the algorithm immediately knows the current partial assignment cannot lead to a solution and backtracks right away, without wasting time trying to assign the emptied-domain exam and failing later. More advanced propagation such as arc-consistency (AC-3) goes further, propagating domain reductions across chains of related exams, shrinking the search space substantially before deep search is even attempted. This significantly reduces the number of dead-end branches explored, which matters greatly given the very large number of courses and students in a real university.

iv. Real-world challenges and the best strategy for scalability

Real-world challenges include: the sheer scale of variables (hundreds of courses, thousands of students) which makes the constraint graph extremely dense, since many courses share students through electives and combined programs; conflicting hard and soft constraints, where satisfying every student-clash and hall constraint perfectly may be infeasible, requiring soft-constraint relaxation and optimization (e.g., minimizing the number of students with back-to-back exams rather than treating it as a hard rule); dynamic changes such as late course additions, faculty unavailability, or hall closures that require efficient re-scheduling without recomputing the entire timetable; and fairness/accommodation requirements that must never be violated even under time pressure.

For scalability, the recommended strategy is backtracking search enhanced with constraint propagation (forward checking or arc-consistency) combined with intelligent variable- and value-ordering heuristics — specifically, the Minimum Remaining Values (MRV) heuristic to first assign the most constrained exams (those with the fewest legal slot/hall options, such as large-enrolment courses or precedence-heavy subjects), and Least Constraining Value (LCV) to choose slot/hall assignments that leave the most flexibility for remaining exams. This combination keeps the search efficient and scalable as the number of courses and students grows, while constraint propagation prunes the search space early, together making automated timetable generation computationally tractable for a large university.

Question 5 — Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. How the Minimax algorithm models this problem

Minimax models the game as a tree of alternating decision layers between two players: the MAX player (the AI, trying to maximize its outcome/score) and the MIN player (the human opponent, assumed to play optimally to minimize the AI's outcome). Each node in the tree represents a game state, and each edge represents a legal move. At terminal nodes (or at a fixed search depth), a static evaluation function assigns a numeric value representing how favourable that state is for the AI.

The algorithm works backward from these leaf values: at MIN nodes, it propagates up the minimum value among children (assuming the opponent picks the worst outcome for the AI), and at MAX nodes, it propagates up the maximum value among children (the AI picks its best available option). This recursive back-propagation gives the AI, at the root, the move that leads to the best guaranteed outcome assuming the opponent also plays optimally — precisely modelling the adversarial, competitive nature of the real-time strategy game.

ii. Computational challenges of large game trees

The central challenge is combinatorial explosion: if the branching factor (average number of legal moves per turn) is b and the search depth is d , the number of nodes Minimax must evaluate grows as $O(b^d)$. In a real-time strategy game, the branching factor can be very large because many units can each take several possible actions simultaneously, so even a modest search depth quickly becomes computationally intractable within the strict per-move time limits available.

This creates a direct tension between search depth (which improves the quality/foresight of decisions) and response time (which is constrained by real-time gameplay requirements). Naive full-depth Minimax is simply too slow, and evaluating every possible future state also consumes excessive memory. This motivates both the pruning techniques discussed next and the use of depth-limited search combined with strong heuristic evaluation functions.

iii. How Alpha-Beta pruning improves efficiency, with detailed reasoning

Alpha-Beta pruning augments Minimax with two bounds carried through the recursion: alpha, the best (highest) value the MAX player can currently guarantee so far along the path to the root, and beta, the best (lowest) value the MIN player can currently guarantee so far. As the tree is explored depth-first, whenever a node's value is found to be worse than what one of the players could already guarantee elsewhere, the remaining sibling branches under that node are 'pruned' (skipped) because a rational opponent would never allow the game to reach that branch in the first place.

Concretely, at a MIN node, if a candidate value is found that is less than or equal to alpha (the MAX player's already-guaranteed best), further exploration of that MIN node's remaining children stops, since MAX would never choose the move leading here. Symmetrically, at a MAX node, if a candidate value is greater than or equal to beta, exploration stops because MIN would never allow this branch to be reached. This pruning does not change the final decision at the root — it is mathematically guaranteed to produce the exact same result as full Minimax — but it

can reduce the effective branching factor dramatically. With a good move-ordering strategy (evaluating likely-best moves first), Alpha-Beta pruning can reduce the effective search complexity from $O(b^d)$ to close to $O(b^{(d/2)})$, roughly doubling the feasible search depth within the same time budget, which is highly valuable for a strict real-time constraint.

iv. Designing an evaluation function and justifying its factors

Since the full game tree cannot be searched to completion in real time, the AI must use a depth-limited search combined with a static evaluation function applied at the cut-off depth. For a real-time strategy game, a well-designed evaluation function should combine multiple weighted factors:

- Material/resource balance: number and strength of units/structures owned by the AI versus the opponent, since army and economic strength are core determinants of long-term advantage.
- Positional advantage: control of strategically valuable map locations (chokepoints, resource nodes, high ground), since position often predicts future tactical opportunities.
- Economic output: resource-gathering rate or production capacity, since RTS outcomes are strongly influenced by sustainable economic growth, not just immediate combat strength.
- Threat/vulnerability assessment: exposure of the AI's key assets to attack versus the opponent's exposure, capturing short-term tactical risk.
- Mobility/tempo: available actions or unit mobility, reflecting the flexibility to respond to the opponent's future moves.

These factors are justified because they collectively approximate the true long-term win probability of a state without requiring the AI to search all the way to the actual end of the game, and because they can each be computed quickly from the current game state, keeping the evaluation function itself cheap enough to be called at every leaf node of a large pruned tree.

v. Strategy for balancing optimality and real-time performance

The recommended strategy is iterative deepening Alpha-Beta search: the AI runs Alpha-Beta Minimax at increasing depth limits (1, 2, 3, ...) within the available time budget, always keeping the best move found at the deepest fully-completed depth, and returning that move immediately when the time limit is reached. This guarantees the AI always has a legal, reasonably good move ready regardless of how much time is available, while still exploiting any extra time to search deeper and improve decision quality.

This is combined with move-ordering heuristics (trying previously good or heuristically promising moves first to maximize pruning efficiency), transposition tables (caching previously evaluated states reached via different move orders, which is common in RTS games), and a well-tuned evaluation function as described above. Together, these techniques let the AI balance the competing goals of optimal/high-quality decision-making and strict real-time responsiveness, ensuring competitive play against human opponents without violating move-time constraints.